# Estratégia de embeddings

Representação vetorial e recuperação dos documentos no RAG

## 1. Objetivo

Embeddings são representações numéricas de textos. No RAG, eles permitem comparar uma pergunta com os trechos dos documentos indexados e recuperar aqueles que possuem maior proximidade semântica. O projeto utiliza essa técnica para localizar dispositivos legais e outros conteúdos textuais relacionados à pergunta, mesmo quando a redação da pergunta não repete exatamente as palavras do documento.

## 2. Modelo utilizado

O projeto utiliza o modelo **intfloat/multilingual-e5-small**, disponibilizado pela família E5 e compatível com textos em diferentes idiomas, incluindo português. A escolha permite manter um modelo relativamente leve para gerar embeddings localmente, sem depender de uma API externa para essa etapa.

O modelo produz vetores com **384 dimensões**. Essa dimensão é verificada durante a inicialização: se a dimensão retornada pelo modelo não corresponder à configuração do banco, a aplicação interrompe a operação para evitar armazenar vetores incompatíveis.

## 3. Prefixos de consulta e documento

A família E5 utiliza prefixos para distinguir os textos que serão indexados das consultas feitas pelos usuários. O projeto aplica esses prefixos de forma consistente:

**passage:** aplicado aos trechos dos documentos antes da geração dos embeddings.

**query:** aplicado às perguntas dos usuários antes da busca vetorial.

Essa distinção faz parte do modo de treinamento do modelo. O código centraliza a aplicação dos prefixos nas funções de geração de embeddings, reduzindo o risco de que sejam esquecidos ou utilizados de forma incorreta.

## 4. Normalização e comparação

Os embeddings são gerados com normalização habilitada. Isso transforma os vetores em vetores unitários, permitindo que a similaridade por cosseno seja utilizada de maneira consistente. No banco PostgreSQL com pgvector, a busca ordena os trechos pela distância de cosseno entre o vetor da pergunta e os vetores armazenados.

A normalização não altera o conteúdo do texto; ela padroniza a escala dos vetores para a comparação. Os embeddings dos documentos são calculados em lotes, com tamanho padrão de 32 textos por lote, o que organiza o processamento durante a ingestão.

## 5. Busca híbrida

A recuperação não depende apenas da similaridade vetorial. O projeto combina duas formas de busca: a busca semântica, baseada nos embeddings, e a busca textual, baseada nos termos presentes nos documentos. A busca textual utiliza o mecanismo Full Text Search do PostgreSQL.

Os resultados das duas buscas são combinados pelo método Reciprocal Rank Fusion (RRF). Em vez de comparar diretamente as pontuações produzidas por mecanismos diferentes, o RRF considera a posição de cada resultado nas listas. Trechos bem posicionados em ambas as buscas tendem a receber uma pontuação combinada maior.

## 6. Fluxo de processamento

Na ingestão, cada chunk é prefixado com **passage:**, convertido em embedding e armazenado no PostgreSQL junto ao texto e aos metadados do documento. Na consulta, a pergunta recebe o prefixo **query:**, é convertida em embedding e utilizada na busca. A recuperação híbrida combina os resultados semânticos e textuais e entrega os trechos selecionados à etapa de geração da resposta.

## Síntese

A estratégia utiliza o multilingual-e5-small com prefixos distintos para documentos e consultas, vetores normalizados e comparação por cosseno. A busca híbrida complementa a recuperação semântica com correspondência textual, combinando os resultados por RRF. Essa abordagem procura equilibrar a identificação de significado com a localização de termos jurídicos específicos.